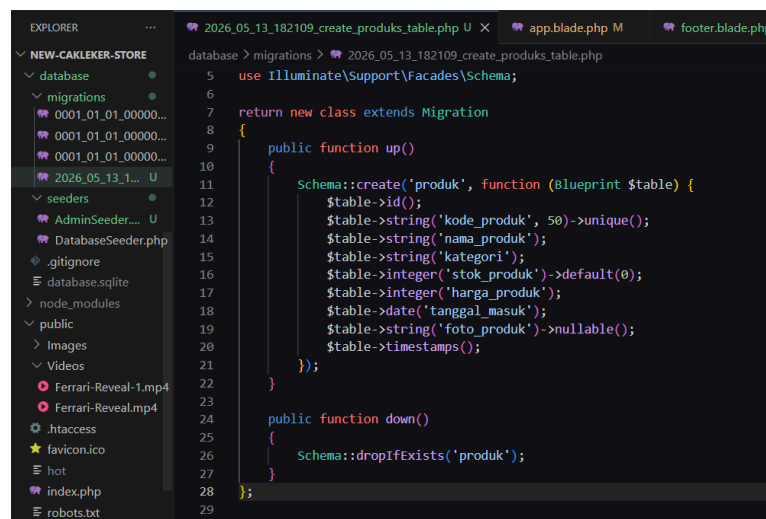


LAPORAN PEMBUATAN PROJECT MENGGUNAKAN LARAVEL BERDASARKAN INSTRUKSI AKTIVITAS MANDIRI 5 DAN 6

Nama : Rafi Hasyadinata
Kelas : Pemrograman Berbasis Web - A
NIM : 242410101019
Nama Proyek : Cakleker Store

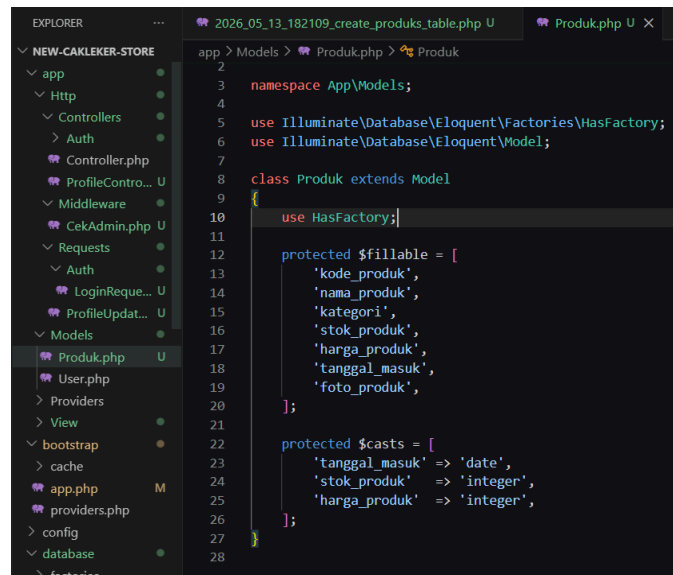
A. Aktivitas Mandiri 5 (Menggunakan Laragon)

1. Membuuat Resource Controller: php artisan make:controller ProdukController --resource --model=Produk



The screenshot shows the VS Code interface with the Explorer panel on the left displaying the file structure of a project named 'NEW-CAKLEKER-STORE'. The 'migrations' folder is expanded, showing several files. The main editor displays the content of the file '2026_05_13_182109_create_produk_table.php'. The code defines a migration class that creates a 'produk' table with columns: id, kode_produk, nama_produk, kategori, stok_produk, harga_produk, tanggal_masuk, and foto_produk.

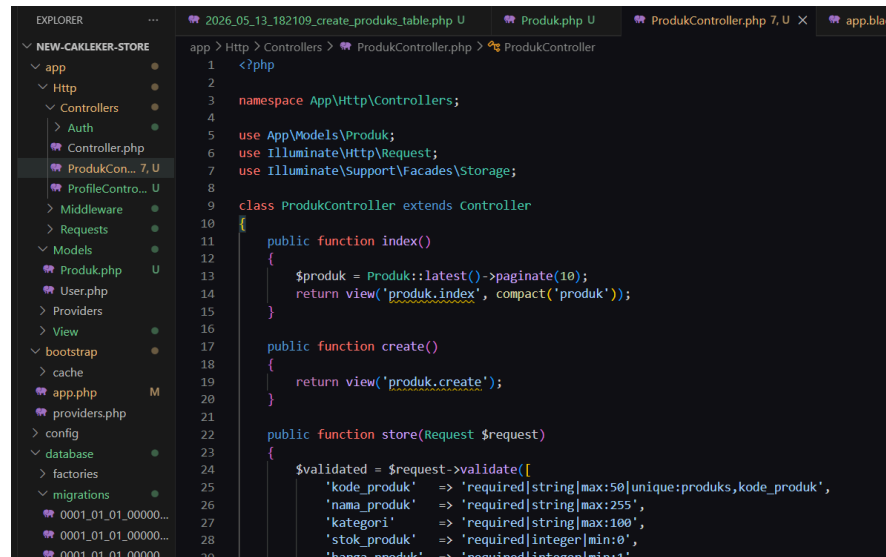
```
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     public function up()
10     {
11         Schema::create('produk', function (Blueprint $table) {
12             $table->id();
13             $table->string('kode_produk', 50)->unique();
14             $table->string('nama_produk');
15             $table->string('kategori');
16             $table->integer('stok_produk')->default(0);
17             $table->integer('harga_produk');
18             $table->date('tanggal_masuk');
19             $table->string('foto_produk')->nullable();
20             $table->timestamps();
21         });
22     }
23
24     public function down()
25     {
26         Schema::dropIfExists('produk');
27     }
28
29 }
```



The screenshot shows the VS Code interface with the Explorer panel on the left displaying the file structure. The 'Models' folder is expanded, showing 'Produk.php'. The main editor displays the content of 'Produk.php', which defines a 'Produk' model class extending 'Model' from 'Illuminate\Database\Eloquent'. The model has fillable fields and casts for dates and integers.

```
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7
8 class Produk extends Model
9 {
10     use HasFactory;
11
12     protected $fillable = [
13         'kode_produk',
14         'nama_produk',
15         'kategori',
16         'stok_produk',
17         'harga_produk',
18         'tanggal_masuk',
19         'foto_produk',
20     ];
21
22     protected $casts = [
23         'tanggal_masuk' => 'date',
24         'stok_produk' => 'integer',
25         'harga_produk' => 'integer',
26     ];
27
28 }
```

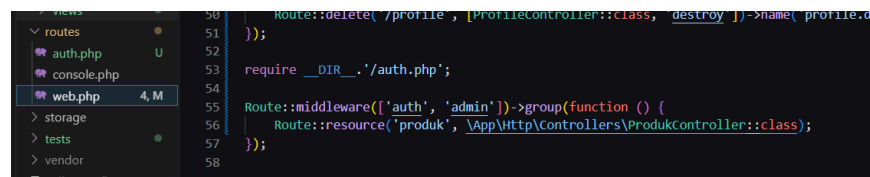
```
PS C:\laragon\www\new-cakleker-store> php artisan make:controller ProdukController --resource --model=Produk
INFO Controller [C:\laragon\www\new-cakleker-store\app\Http\Controllers\ProdukController.php] created successfully.
```



```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Models\Produk;
6 use Illuminate\Http\Request;
7 use Illuminate\Support\Facades\Storage;
8
9 class ProdukController extends Controller
10 {
11     public function index()
12     {
13         $produk = Produk::latest()->paginate(10);
14         return view('produk.index', compact('produk'));
15     }
16
17     public function create()
18     {
19         return view('produk.create');
20     }
21
22     public function store(Request $request)
23     {
24         $validated = $request->validate([
25             'kode_produk' => 'required|string|max:50|unique:produks,kode_produk',
26             'nama_produk' => 'required|string|max:255',
27             'kategori' => 'required|string|max:100',
28             'stok_produk' => 'required|integer|min:0',
29             'harga_produk' => 'required|integer|min:1'
```

Perintah php artisan make:controller ProdukController --resource --model=Produk akan membuat file controller ProdukController di app/Http/Controllers yang sudah berisi method resource (index, create, store, edit, update, destroy).

2. Mendaftarkan di routes/web.php: Route::resource('mahasiswa', MahasiswaController::class)



```
50 Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
51
52
53 require __DIR__.'/auth.php';
54
55 Route::middleware(['auth', 'admin'])->group(function () {
56     Route::resource('produk', \App\Http\Controllers\ProdukController::class);
57 });
58
```

Dengan menambahkan Route::resource('produk', ProdukController::class) di routes/web.php, Laravel secara otomatis membuat tujuh route untuk operasi CRUD: GET /produk (index), GET /produk/create (create), POST /produk (store), GET /produk/{produk} (show), GET /produk/{produk}/edit (edit), PUT/PATCH /produk/{produk} (update), DELETE /produk/{produk} (destroy). Kita akan tempatkan route ini di dalam grup middleware auth dan admin agar hanya admin yang bisa mengelola produk.

3. Membuat semua view: mahasiswa/index.blade.php (tabel + pagination), create.blade.php (form), edit.blade.php (form pre-filled), show.blade.php (detail)

```
EXPLORER  ...  web.php M  index.blade.php X  2026_05_13_182109_create_produk_table.php U  Produk.php U  ProdukController.php U  app.blade.php M

NEW-CAKLEKER-STORE
  resources
  views
  auth
  components
  layouts
  partials
  produk
  index.blade.p... U
  profile
  dashboard.bla... M
  tentang.blade... M
  welcome.blad... M
  routes
  auth.php U
  console.php
  web.php M
  storage
  tests
  vendor
  editorconfig
  env
  env.example
  gitattributes
  gitignore
  ipnrc
  artisan
  composer.json M
  composer.lock M
  LAPORAN AKTIVITAS ...
  LAPORAN AKTIVITAS ...
  package-lock.json U
  package.json M
  phpunit.xml
  postcss.config.js U

resources > views > produk > index.blade.php > ...
1 @extends('layouts.app')
2 @section('title', 'Daftar Produk')
3 @section('content')
4 <div class="container">
5 <h1 class="mb-4">Daftar Produk</h1>
6 <a href="{{ route('produk.create') }}" class="btn btn-primary mb-3">Tambah Produk</a>
7 @if(session('success'))
8 <div class="alert alert-success">{{ session('success') }}</div>
9 @endif
10 <table class="table table-dark table-striped">
11 <thead>
12 <tr><th>kode</th><th>nama</th><th>kategori</th><th>stok</th><th>harga</th><th>aksi</th></tr>
13 </thead>
14 <tbody>
15 @foreach($produk as $item)
16 <tr>
17 <td>{{ $item->kode_produk }}</td>
18 <td>{{ $item->nama_produk }}</td>
19 <td>{{ $item->kategori }}</td>
20 <td>{{ $item->stok }}</td>
21 <td>Rp {{ number_format($item->harga, 0, ',', '.') }}</td>
22 <td>
23 <a href="{{ route('produk.show', $item->id) }}" class="btn btn-sm btn-info">Detail</a>
24 <a href="{{ route('produk.edit', $item->id) }}" class="btn btn-sm btn-warning">Edit</a>
25 <form action="{{ route('produk.destroy', $item->id) }}" method="POST" style="display:inline;">
26 @csrf @method('DELETE')
27 <button type="submit" class="btn btn-sm btn-danger" onclick="return confirm('Yakin hapus?')">Hapus</button>
28 </td>
29 </tr>
30 @endforeach
31 </tbody>
32 </table>
33 @foreach($produk->links() )
34 </div>
35 @endsection
36
37
```

```
EXPLORER  ...  web.php M  index.blade.php U  create.blade.php X  2026_05_13_182109_create_produk_table.php U  Produk.php U  ProdukController.php U

NEW-CAKLEKER-STORE
  resources
  views
  auth
  components
  layouts
  partials
  produk
  create.blade.p... U
  index.blade.p... U
  profile
  dashboard.bla... M
  tentang.blade... M
  welcome.blad... M
  routes
  auth.php U
  console.php
  web.php M
  storage
  tests

resources > views > produk > create.blade.php > ...
1 @extends('layouts.app')
2 @section('title', 'Tambah Produk')
3 @section('content')
4 <div class="container">
5 <h1>Tambah Produk Baru</h1>
6 <form action="{{ route('produk.store') }}" method="POST" enctype="multipart/form-data">
7 @csrf
8 <div class="mb-3"><label>Kode Produk</label><input type="text" name="kode_produk" class="form-control" required</div>
9 <div class="mb-3"><label>Nama Produk</label><input type="text" name="nama_produk" class="form-control" required</div>
10 <div class="mb-3"><label>Kategori</label><input type="text" name="kategori" class="form-control" required</div>
11 <div class="mb-3"><label>Stok</label><input type="number" name="stok" class="form-control" required</div>
12 <div class="mb-3"><label>Harga</label><input type="number" name="harga" class="form-control" required</div>
13 <div class="mb-3"><label>Deskripsi</label><input type="text" name="deskripsi" class="form-control" required</div>
14 <div class="mb-3"><label>Foto</label><input type="file" name="foto" class="form-control"></div>
15 <div class="mb-3"><button type="submit" class="btn btn-primary">Simpan</button>
16 </form>
17 </div>
18 @endsection
19
```

```
EXPLORER  ...  web.php M  index.blade.php U  create.blade.php U  edit.blade.php X  2026_05_13_182109_create_produk_table.php U  Produk.php U  ProdukContr D

NEW-CAKLEKER-STORE
  resources
  views
  auth
  components
  layouts
  partials
  produk
  create.blade.p... U
  edit.blade.php U
  index.blade.p... U
  profile
  dashboard.bla... M
  tentang.blade... M
  welcome.blad... M
  routes
  auth.php U
  console.php
  web.php M
  storage
  tests

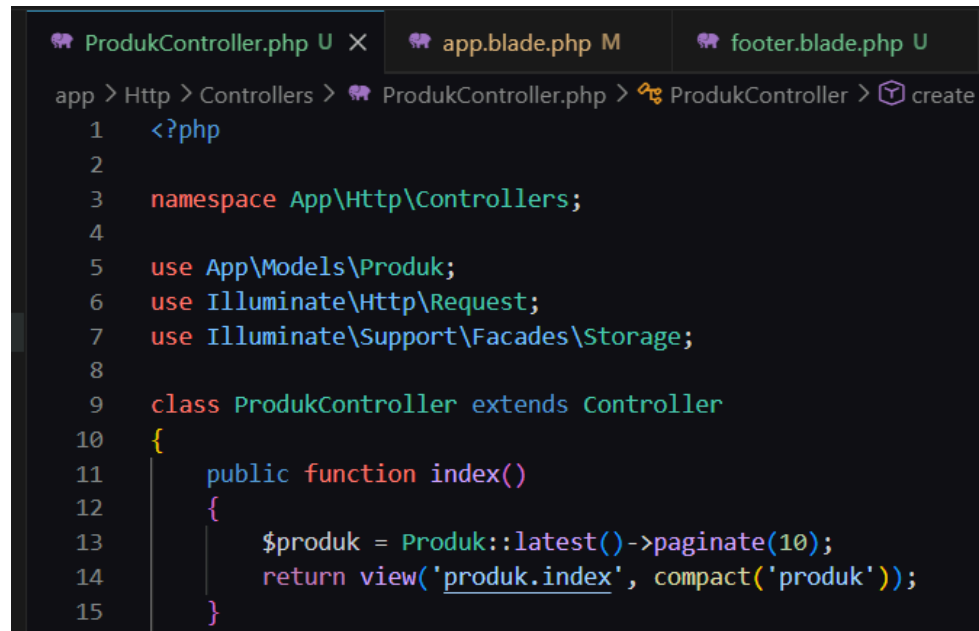
resources > views > produk > edit.blade.php > ...
1 @extends('layouts.app')
2 @section('title', 'Edit Produk')
3 @section('content')
4 <div class="container">
5 <h1>Edit Produk</h1>
6 <form action="{{ route('produk.update', $produk->id) }}" method="POST" enctype="multipart/form-data">
7 @csrf @method('PUT')
8 <div class="mb-3"><label>Kode Produk</label><input type="text" name="kode_produk" value="{{ old('kode_produk', $produk->kode_produk) }}" cla
9 <div class="mb-3"><label>Nama Produk</label><input type="text" name="nama_produk" value="{{ old('nama_produk', $produk->nama_produk) }}" cla
10 <div class="mb-3"><label>Kategori</label><input type="text" name="kategori" value="{{ old('kategori', $produk->kategori) }}" class="form-con
11 <div class="mb-3"><label>Stok</label><input type="number" name="stok" value="{{ old('stok', $produk->stok) }}" class="form-control" required
12 <div class="mb-3"><label>Harga</label><input type="number" name="harga" value="{{ old('harga', $produk->harga) }}" class="form-control" requi
13 <div class="mb-3"><label>Deskripsi</label><input type="text" name="deskripsi" value="{{ old('deskripsi', $produk->deskripsi) }}" class="form-control" required
14 <div class="mb-3"><label>Foto</label><input type="file" name="foto" class="form-control"></div>
15 <div class="mb-3"><button type="submit" class="btn btn-primary">Update</button>
16 </form>
17 </div>
18 @endsection
19
```

```
EXPLORER  ...  web.php M  index.blade.php U  create.blade.php U  edit.blade.php U  show.blade.php X  2026_05_13_182109_create_produk_table.php U  Produk.php U  ProdukContr D

NEW-CAKLEKER-STORE
  resources
  views
  auth
  components
  layouts
  partials
  produk
  edit.blade.php U
  index.blade.p... U
  show.blade.p... U
  profile
  dashboard.bla... M
  tentang.blade... M
  welcome.blad... M
  routes
  auth.php U
  console.php
  web.php M
  storage
  tests

resources > views > produk > show.blade.php > ...
1 @extends('layouts.app')
2 @section('title', 'Detail Produk')
3 @section('content')
4 <div class="container">
5 <h1>Detail Produk</h1>
6 <p><strong>Kode:</strong> {{ $produk->kode_produk }}</p>
7 <p><strong>Nama:</strong> {{ $produk->nama_produk }}</p>
8 <p><strong>Kategori:</strong> {{ $produk->kategori }}</p>
9 <p><strong>Stok:</strong> {{ $produk->stok }}</p>
10 <p><strong>Harga:</strong> Rp {{ number_format($produk->harga, 0, ',', '.') }}</p>
11 <p><strong>Deskripsi:</strong> {{ $produk->deskripsi }}</p>
12 @if($produk->foto) 
13 <br><a href="{{ route('produk.index') }}" class="btn btn-secondary mt-3">Kembali</a>
14 </div>
15 @endsection
16
```

- Implementasi method index() — ambil semua produk dengan paginate(10) dan tampilkan ke view



```
ProdukController.php U X app.blade.php M footer.blade.php U
app > Http > Controllers > ProdukController.php > ProdukController > create
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Produk;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Storage;
8
9  class ProdukController extends Controller
10 {
11     public function index()
12     {
13         $produk = Produk::latest()->paginate(10);
14         return view('produk.index', compact('produk'));
15     }
16 }
```

Method index() mengambil semua produk, diurutkan misalnya berdasarkan created_at desc, lalu paginate(10). Data dikirim ke view.

- Implementasi create() dan store() dengan Validasi

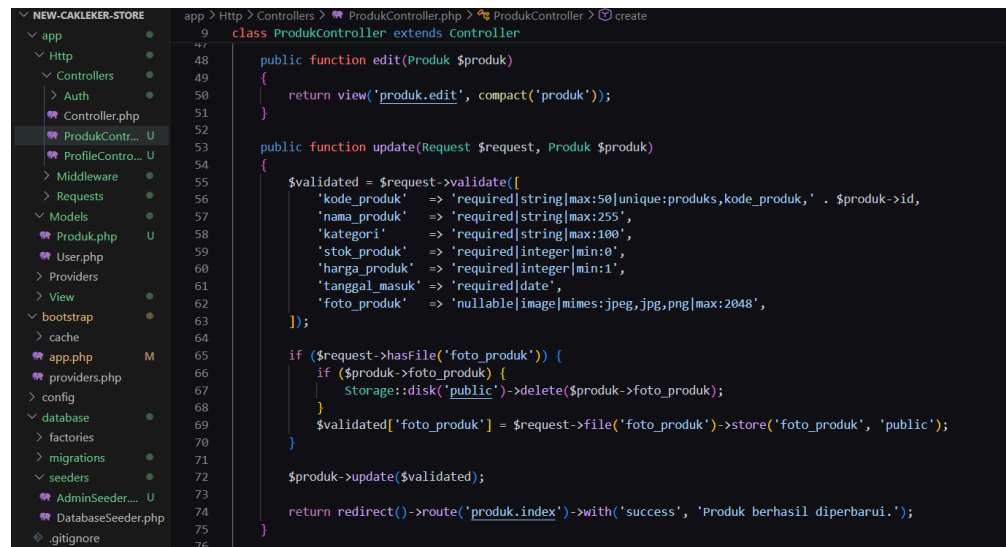


```
> Auth 17
> Controller.php 18
> ProdukContr... U 19
> ProfileContr... U 20
> Middleware 21
> Requests 22
> Models 23
> Produk.php U 24
> User.php 25
> Providers 26
> View 27
> bootstrap 28
> cache 29
> app.php M 30
> providers.php 31
> config 32
> database 33
> factories 34
> migrations 35
> seeders 36
> AdminSeeder... U 37
> DatabaseSeeder.php 38

17 public function create()
18 {
19     return view('produk.create');
20 }
21
22 public function store(Request $request)
23 {
24     $validated = $request->validate([
25         'kode_produk' => 'required|string|max:50|unique:produk,kode_produk',
26         'nama_produk' => 'required|string|max:255',
27         'kategori' => 'required|string|max:100',
28         'stok_produk' => 'required|integer|min:0',
29         'harga_produk' => 'required|integer|min:1',
30         'tanggal_masuk' => 'required|date',
31         'foto_produk' => 'nullable|image|mimes:jpeg,jpg,png|max:2048',
32     ]);
33
34     if ($request->hasFile('foto_produk')) {
35         $validated['foto_produk'] = $request->file('foto_produk')->store('foto_produk', 'public');
36     }
37
38     Produk::create($validated);
39
40     return redirect()->route('produk.index')->with('success', 'Produk berhasil ditambahkan.');
```

create() menampilkan form. store() melakukan validasi (kode_produk unik, stok integer >=0, harga >=1, tanggal_masuk format date, foto mimes:jpg/png max 2MB), menyimpan data, dan upload file foto ke storage/app/public/foto_produk.

6. Implementasi edit() dan update()



```
class ProdukController extends Controller
{
    public function edit(Produk $produk)
    {
        return view('produk.edit', compact('produk'));
    }

    public function update(Request $request, Produk $produk)
    {
        $validated = $request->validate([
            'kode_produk' => 'required|string|max:50|unique:produks,kode_produk,' . $produk->id,
            'nama_produk' => 'required|string|max:255',
            'kategori' => 'required|string|max:100',
            'stok_produk' => 'required|integer|min:0',
            'harga_produk' => 'required|integer|min:1',
            'tanggal_masuk' => 'required|date',
            'foto_produk' => 'nullable|image|mimes:jpeg,jpg,png|max:2048',
        ]);

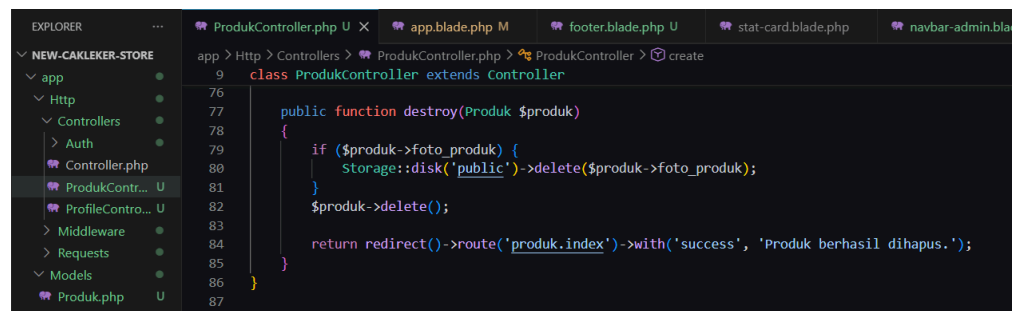
        if ($request->hasFile('foto_produk')) {
            if ($produk->foto_produk) {
                Storage::disk('public')->delete($produk->foto_produk);
            }
            $validated['foto_produk'] = $request->file('foto_produk')->store('foto_produk', 'public');
        }

        $produk->update($validated);

        return redirect()->route('produk.index')->with('success', 'Produk berhasil diperbarui.');
```

edit() menampilkan form dengan data produk. update() memvalidasi, dengan aturan unique pada kode_produk yang mengecualikan ID saat ini. Jika ada upload foto baru, hapus foto lama dan simpan yang baru.

7. Implementasi destroy() — dengan konfirmasi JavaScript sebelum hapus



```
public function destroy(Produk $produk)
{
    if ($produk->foto_produk) {
        Storage::disk('public')->delete($produk->foto_produk);
    }
    $produk->delete();

    return redirect()->route('produk.index')->with('success', 'Produk berhasil dihapus.');
```

destroy() menghapus produk dan file fotonya. Di view, tombol hapus sudah memiliki onclick="return confirm('Yakin?')".

8. Menambahkan flash message di layout dan tampilkan session('success') / session('error')



```
<main>
    @if(session('success'))
        <div class="alert-success" style="background: #4CAF50; color: white; padding: 10px; border-radius: 5px; margin-bottom: 20px;">{{ session('success') }}</div>
    @endif
    @if(session('error'))
        <div class="alert-error" style="background: #f44336; color: white; padding: 10px; border-radius: 5px; margin-bottom: 20px;">{{ session('error') }}</div>
    @endif
    @yield('content')
</main>
```

9. Menambahkan fitur upload foto produk (sudah dimasukkan di method store() dan update ())

```
'kategori' => 'required|string|max:100',
'stok_produk' => 'required|integer|min:0',
'harga_produk' => 'required|integer|min:1',
'tanggal_masuk' => 'required|date',
'foto_produk' => 'nullable|image|mimes:jpeg,jpg,png|max:2048',
]);

if ($request->hasFile('foto_produk')) {
    $validated['foto_produk'] = $request->file('foto_produk')->store('foto_produk', 'public');
}
```

Upload foto sudah diintegrasikan dalam method store dan update. Validasi memastikan file yang diupload adalah gambar (jpeg/png) dengan ukuran maksimal 2MB. File akan disimpan di direktori storage/app/public/produk menggunakan sistem file Laravel. Untuk mengakses foto dari browser, kita perlu menjalankan php artisan storage:link yang membuat symbolic link dari public/storage ke storage/app/public. Foto yang sudah diupload akan ditampilkan di halaman edit dan show.

B. Aktivitas Mandiri 6 (Menggunakan Laragon)

1. Menginstal Laravel Breeze

```
PS C:\laragon\www\new-cakleker-store> composer require laravel/breeze --dev
./composer.json has been updated
Running composer update laravel/breeze
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking laravel/breeze (v2.4.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading laravel/breeze (v2.4.1)
  - Installing laravel/breeze (v2.4.1): Extracting archive
Generating optimized autoload files
> illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

 INFO  Discovering packages.

laravel/breeze ..... DONE
laravel/pail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

78 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

 INFO  No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^2.4 for laravel/breeze
```

```
PS C:\laragon\www\new-cakleker-store> php artisan breeze:install blade

 INFO  Installing and building Node dependencies.

added 139 packages, and audited 140 packages in 45s

34 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities

> build
> vite build

vite v6.0.12 building client environment for production...
transforming... ✓ 3 modules transformed.
x Build failed in 11.36s
error during build:
Build failed with 1 error:

[UNRESOLVED_IMPORT] Error: Could not resolve './bootstrap' in resources/js/app.js
  [ resources/js/app.js:1:8 ]
    1 | import './bootstrap';
      |           ~~~~~ Module not found.

at aggregateBindingErrorsInto$Error (file:///C:/laragon/www/new-cakleker-store/node_modules/rollup/dist/shared/error-BmYQ-g.mjs:48:18)
at transform$1 (file:///C:/laragon/www/new-cakleker-store/node_modules/rollup/dist/shared/transform$1-BmYQ-g.mjs:118:128)
at build (file:///C:/laragon/www/new-cakleker-store/node_modules/rollup/dist/shared/rollup-build-D_Sht11.mjs:3257:34)
at async buildEnvironment (file:///C:/laragon/www/new-cakleker-store/node_modules/vite/dist/node/chunks/node.js:33052:64)
at async Object.build (file:///C:/laragon/www/new-cakleker-store/node_modules/vite/dist/node/chunks/node.js:33474:19)
at async Object.build$1 (file:///C:/laragon/www/new-cakleker-store/node_modules/vite/dist/node/chunks/node.js:33471:153)
at async CAC.cannyous$ (file:///C:/laragon/www/new-cakleker-store/node_modules/vite/dist/node/cli.js:778:3) {
  errors: [Getter/Setter]
}

 INFO  Breeze scaffolding installed successfully.
```

Laravel Breeze menyediakan komponen autentikasi berbasis Blade (login, register, lupa password, reset password). Perintah breeze:install blade akan menyalin file view, controller, dan rute autentikasi. Di situ terdapat error terkait tidak ditemukannya modul bootstrap di file [app.js](#) di folder resources/js, tetapi tidak mempengaruhi fungsi utama Laravel Breeze nantinya.

2. Menjalankan migration tabel users

```
PS C:\laragon\www\new-cakleker-store> php artisan migrate
INFO: Preparing database.
Creating migration table ..... 183.86ms DONE
INFO: Running migrations.
0001_01_01_000000_create_users_table ..... 218.81ms DONE
0001_01_01_000001_create_cache_table ..... 134.46ms DONE
0001_01_01_000002_create_jobs_table ..... 147.44ms DONE
PS C:\laragon\www\new-cakleker-store>
```

Migrasi akan membuat tabel users, password_reset_tokens, sessions, dan cache. Pastikan file .env sudah dikonfigurasi dengan database yang benar (misal DB_DATABASE=new_cakleker_store_db).

3. Menjalankan npm install && npm run dev

```
PS C:\laragon\www\new-cakleker-store> npm install
npm : File C:\laragon\bin\nodejs\node-v22\npm.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm install
+ ~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\laragon\www\new-cakleker-store>
```

```
C:\laragon\www
λ cd C:\laragon\www\new-cakleker-store

C:\laragon\www\new-cakleker-store(new-cakleker-store -> origin)
λ npm install

up to date, audited 140 packages in 2s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\laragon\www\new-cakleker-store(new-cakleker-store -> origin)
λ npm run dev

> dev
> vite

VITE v8.0.12 ready in 721 ms
  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help

LARAVEL v13.6.0 plugin v3.1.0
  → APP_URL: http://localhost
23.06.44 [vite] (client) [optimizer] bundling dependencies...
```

npm install dan npm run dev digunakan untuk mengkompilasi aset CSS/JS (Tailwind CSS) yang digunakan Breeze. Dikarenakan prosesnya mengalami error terkait perizinan sistem operasi device, maka prosesnya dilakukan di terminal laragon.

4. Mencoba melakukan register dan login

```
PS C:\laragon\www\new-cakleker-store> php artisan make:seeder AdminSeeder
```

```
INFO Seeder [C:\laragon\www\new-cakleker-store\database\seeders\AdminSeeder.php] created successfully.
```

```
PS C:\laragon\www\new-cakleker-store> php artisan db:seed --class=AdminSeeder
```

```
INFO Seeding database.
```

```
database > seeders > AdminSeeder.php > ...
```

```
1  <?php
2
3  namespace Database\Seeders;
4
5  use Illuminate\Database\Seeder;
6  use Illuminate\Support\Facades\Hash;
7  use App\Models\User;
8
9  class AdminSeeder extends Seeder
10 {
11     public function run()
12     {
13         User::create([
14             'name' => 'Admin',
15             'email' => 'admin@cakleker.com',
16             'password' => Hash::make('password'),
17             'role' => 'admin',
18         ]);
19     }
20 }
21
```

```
app > Http > Controllers > Auth > RegisteredUserController.php > RegisteredUserController > store
```

```
1  <?php
2
3  namespace App\Http\Controllers\Auth;
4
5  use App\Http\Controllers\Controller;
6  use App\Models\User;
7  use Illuminate\Auth\Events\Registered;
8  use Illuminate\Http\RedirectResponse;
9  use Illuminate\Http\Request;
10 use Illuminate\Support\Facades\Auth;
11 use Illuminate\Support\Facades\Hash;
12 use Illuminate\Validation\Rules;
13 use Illuminate\View\View;
14
15 class RegisteredUserController extends Controller
16 {
17     public function create(): View
18     {
19         return view('auth.register');
20     }
21
22     public function store(Request $request): RedirectResponse
23     {
24         $request->validate([
25             'name' => ['required', 'string', 'max:255'],
26             'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
27             'password' => ['required', 'confirmed', Rules\Password::defaults()],
28         ]);
29
30         $user = User::create([
31             'name' => $request->name,
32             'email' => $request->email,
33             'password' => Hash::make($request->password),
34             'role' => 'customer',
35         ]);
36
37         event(new Registered($user));
38         Auth::login($user);
39         return redirect(route('dashboard'));
40     }
41 }
42
```

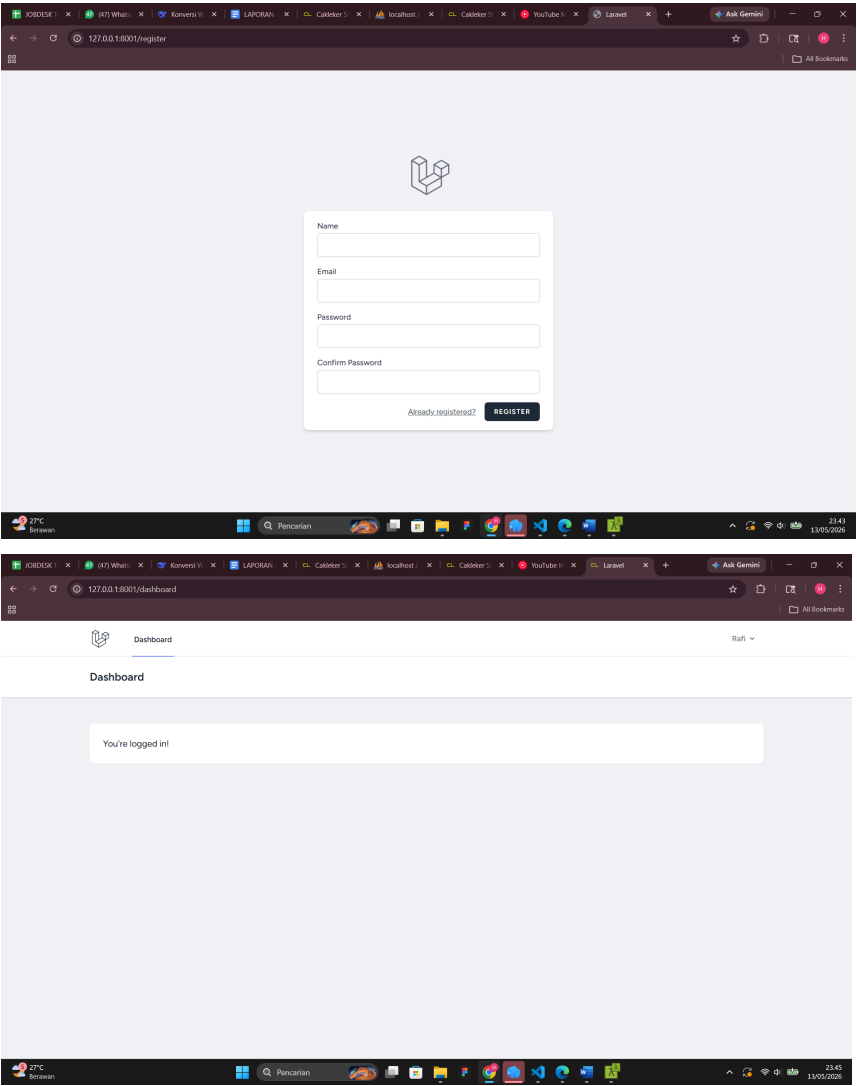


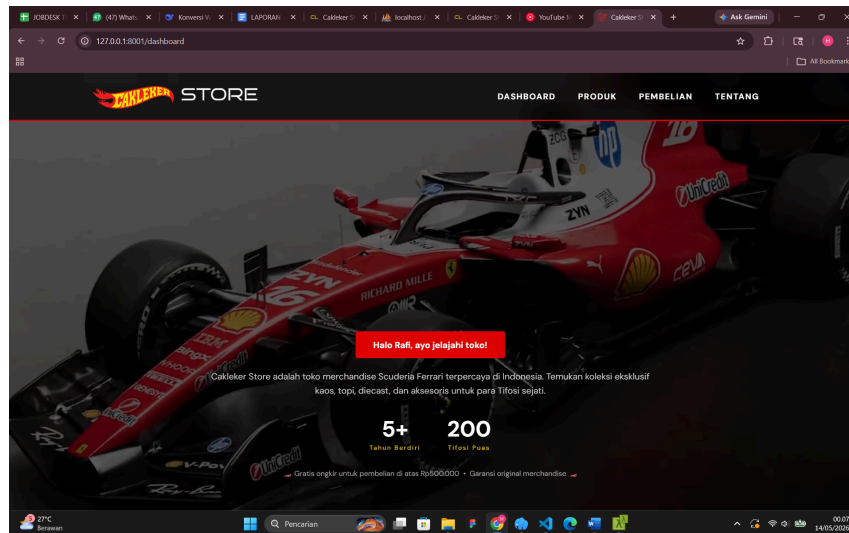
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVDB
PS C:\laragon\www\new-cakleker-store> php artisan serve --port=8001

[INFO] Server running on [http://127.0.0.1:8001].

Press Ctrl+C to stop the server

2026-05-13 23:39:21 / .....
2026-05-13 23:39:22 /favicon.ico .....
2026-05-13 23:40:13 / .....
2026-05-13 23:40:14 /favicon.ico .....
2026-05-13 23:41:56 /register .....
2026-05-13 23:42:00 /favicon.ico .....
```





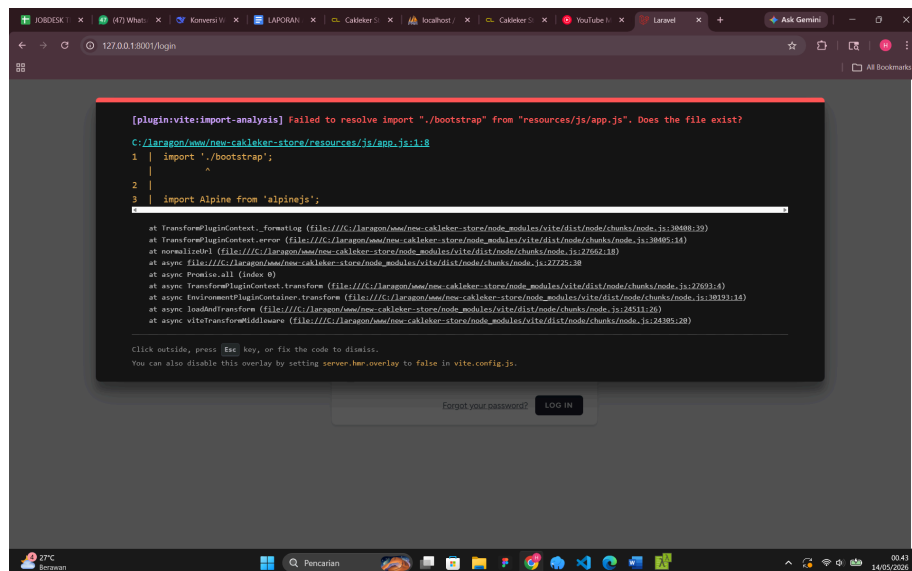
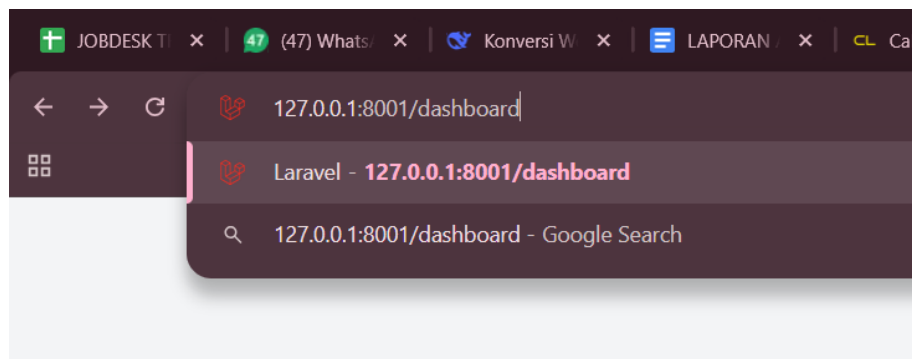
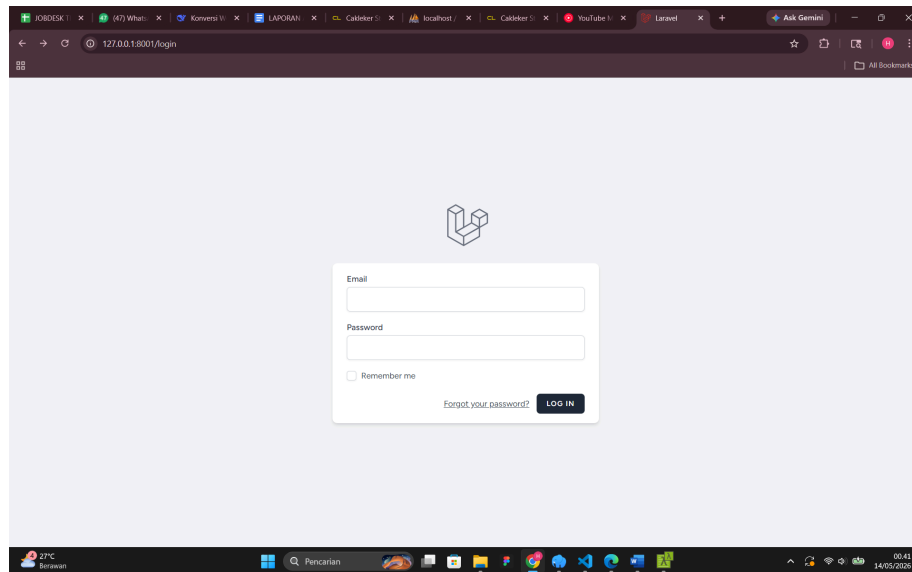
Karena hanya role customer yang dapat melakukan registrasi (admin tidak perlu daftar), jadi diperlukan modifikasi controller RegisteredUserController yang dibuat oleh Breeze. Di dalam method store, nilai kolom role akan dipaksa menjadi 'customer' saat membuat user baru. Dengan demikian, siapapun yang mendaftar akan otomatis menjadi customer. Admin dibuat secara manual melalui seeder.

5. Mengamankan Semua Route Customer dengan Middleware auth

```
Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit']->name('profile.edit'));
    Route::patch('/profile', [ProfileController::class, 'update']->name('profile.update'));
    Route::delete('/profile', [ProfileController::class, 'destroy']->name('profile.destroy'));
});
```

Route yang bersifat customer (seperti halaman produk, pembelian, dashboard, dll) harus dilindungi agar tidak dapat diakses oleh pengguna yang belum login. Dengan mengelompokkan route dalam grup middleware('auth'), Laravel akan secara otomatis memeriksa status autentikasi sebelum mengizinkan akses. Jika pengguna belum login, mereka akan diarahkan ke halaman login.

6. Mencoba mengakses /dashboard (milik customer) tanpa login

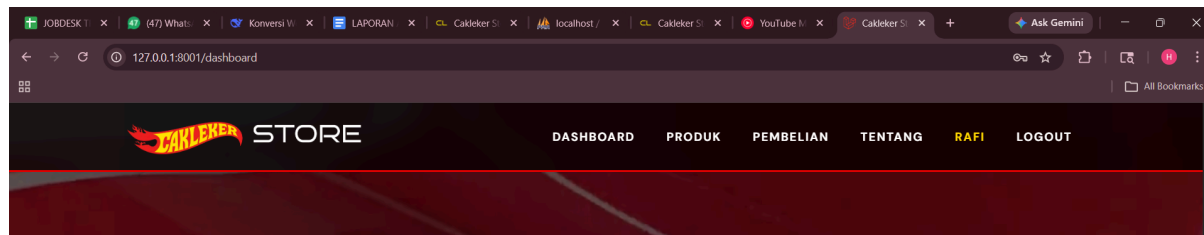


Ketika pengguna yang belum login mencoba mengakses URL /dashboard, sistem akan mendeteksi bahwa tidak ada session autentikasi aktif. Hal ini karena route /dashboard telah dilindungi oleh middleware auth. Middleware tersebut akan memeriksa status login; jika belum login, pengguna secara otomatis akan diarahkan (redirect) ke halaman /login. Mekanisme ini

memastikan bahwa hanya pengguna yang telah terautentikasi yang dapat mengakses halaman dashboard, sehingga meningkatkan keamanan aplikasi.

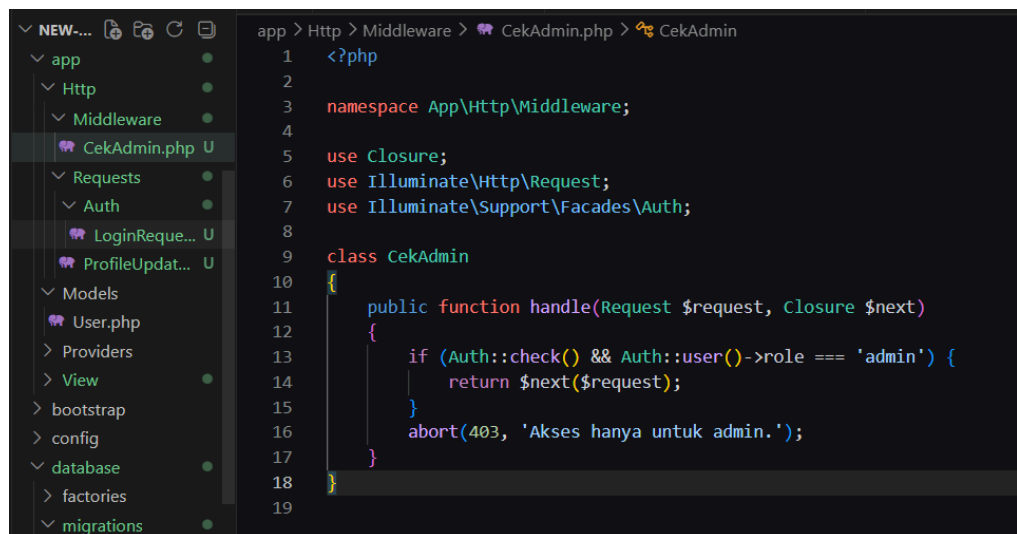
7. Menambahkan nama user di navbar: `{{ auth()->user()->name }}`

```
<a href="{{ route('pembelian') }}">Pembelian</a>
<a href="{{ route('tentang') }}">Tentang</a>
<span class="user-name">{{ auth()->user()->name }}</span>
<form method="POST" action="{{ route('logout') }}" style="display: inline;">
    @csrf
```



8. Buat custom middleware CekAdmin yang hanya mengizinkan user dengan role 'admin' mengakses route tertentu

```
PS C:\laragon\www\new-cakleker-store> php artisan make:middleware CekAdmin
INFO Middleware [C:\laragon\www\new-cakleker-store\app\Http\Middleware\CekAdmin.php] created successfully.
```



```

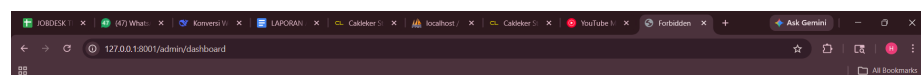
1  <?php
2
3  use Illuminate\Foundation\Application;
4  use Illuminate\Foundation\Configuration\Exceptions;
5  use Illuminate\Foundation\Configuration\Middleware;
6  use App\Http\Middleware\CekAdmin;
7
8  return Application::configure(basePath: dirname(__DIR__))
9      ->withRouting(
10         web: __DIR__.'/../routes/web.php',
11         commands: __DIR__.'/../routes/console.php',
12         health: 'up',
13     )
14     ->withMiddleware(function (Middleware $middleware) {
15         $middleware->alias([
16             'admin' => CekAdmin::class,
17         ]);
18     })
19     ->withExceptions(function (Exceptions $exceptions) {
20         //
21     }->create());
22

```

```

12  class AuthenticatedSessionController extends Controller
13  {
14      /**
15       * Handle an incoming authentication request.
16       */
17      public function store(LoginRequest $request): RedirectResponse
18      {
19          $request->authenticate();
20
21          $request->session()->regenerate();
22
23          $user = Auth::user();
24
25          if ($user->role === 'admin') {
26              return redirect()->intended(route('admin.dashboard'));
27          }
28
29          return redirect()->intended(route('dashboard'));
30      }
31  }
32

```



Pada langkah ini, middleware CekAdmin yang telah didaftarkan di bootstrap/app.php digunakan untuk melindungi rute khusus admin. Rute

/admin/dashboard ditempatkan dalam grup middleware ['auth', 'admin'], sehingga hanya pengguna yang sudah login dan memiliki role admin yang dapat mengaksesnya. Selain itu, untuk mengatur arah redirect setelah login, dilakukan modifikasi pada AuthenticatedSessionController pada method store. Setelah proses autentikasi berhasil dan session diregenerasi, sistem memeriksa role user yang sedang login. Jika role adalah admin, pengguna akan diarahkan ke rute admin.dashboard; jika tidak (yaitu customer), akan diarahkan ke rute dashboard biasa. Dengan demikian, setiap pengguna akan masuk ke halaman yang sesuai dengan perannya. Untuk menguji fungsionalitas ini, dilakukan registrasi customer baru (yang otomatis memiliki role customer) kemudian login, dan dipastikan masuk ke /dashboard. Sementara itu, login dengan akun admin (yang telah dibuat melalui seeder) akan masuk ke /admin/dashboard. Jika customer mencoba mengakses /admin/dashboard secara manual, middleware admin akan menolak akses dan menampilkan error 403. Dengan langkah ini, sistem autentikasi berbasis role sudah berjalan dengan baik, memisahkan area admin dan customer secara aman.

9. Modifikasi index() Controller agar hanya menampilkan data yang dibuat oleh user yang sedang login